SET 4

PART A — Short questions and answers

Q1. Interpret the 1:N cardinality between ROOM and STUDENT in a hostel ER diagram.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Explain the effect of ON DELETE CASCADE on the foreign key StudentID in ALLOCATION.

Answer: A foreign key maintains a valid relationship by referring to a key in another table.

Q3. Explain a partial dependency using the composite key (Student_ID, Room_No).

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Interpret the query: SELECT s.StudentName, r.RoomNo FROM STUDENT s JOIN ALLOCATION a ON s.StudentID = a.StudentID JOIN ROOM r ON a.RoomNo = r.RoomNo;

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Explain the purpose of GROUP BY with COUNT, using Block as the grouping column.

Answer: GROUP BY collects rows with the same value so aggregate functions can calculate a result for each group.

Q6. Describe the use of DBMS_OUTPUT.PUT_LINE.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Describe schema flexibility in document databases compared with relational databases.

Answer: Document databases allow documents to have different fields, making them useful when records vary.

Q8. Explain the relationship between a database, a collection and a document in MongoDB.

Answer: A MongoDB document is one record represented as field–value pairs, similar to a JSON object.

Q9. Interpret the MongoDB query: db.rooms.find({Block: "A", Capacity: {$gte: 3}})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Explain what the following Cypher statement does: CREATE (s:Student {name: 'Ravi'});

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A residential college maintains a Hostel Management System to store student details, room details, and

1(a) ER design:

• **STUDENT**(StudentID, StudentName, Department, Year); primary key: **StudentID**.

• **ROOM**(RoomNo, Block, Capacity, MonthlyRent); primary key: **RoomNo**.

• **ALLOCATION**(AllocationID, StudentID, RoomNo, CheckInDate, CheckOutDate); primary key: **AllocationID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE STUDENT (
  StudentID NUMBER PRIMARY KEY,
  StudentName VARCHAR2(100),
  Department VARCHAR2(100),
  Year VARCHAR2(100)
);
CREATE TABLE ROOM (
  RoomNo VARCHAR2(100) PRIMARY KEY,
  Block VARCHAR2(100),
```

```
  Capacity NUMBER,
  MonthlyRent VARCHAR2(100)
);
CREATE TABLE ALLOCATION (
  AllocationID NUMBER PRIMARY KEY,
  StudentID NUMBER,
  RoomNo VARCHAR2(100),
  CheckInDate DATE,
  CheckOutDate DATE
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM ALLOCATION x
JOIN STUDENT a ON x.StudentID = a.StudentID
JOIN ROOM b ON x.RoomNo = b.RoomNo;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```